

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

Лабораторна робота № 11

з дисципліни «Алгоритмізація та програмування»
на тему «Розробка десктоп-застосунків в середовищі QtCreator»

ХАІ.301. електрична інженерія. Група 319а. номер в списку 10 ЛР

Виконав студент гр. _____319а_____

_____Аріна КУШАКОВА_____
(підпис, дата) (П.І.Б.)

Перевірів

_____к.т.н., доц. Олена ГАВРИЛЕНКО_____
(підпис, дата) (П.І.Б.)

МЕТА РОБОТИ

Ознайомитися з основами розробки програм з використанням QtDesigner і навчитися розробляти десктоп-застосунки із графічним користувацьким інтерфейсом для введення/виведення даних на мові програмування C++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вивчити алгоритм створення проекту Qt Widgets Application в середовищі QtCreator. Ознайомитись з налаштуваннями основних елементів для введення, виведення, компоновки форми і управління. Завдання 2. Для вирішення завдання (табл.1) відповідно до варіанта:

А. Спроектувати і реалізувати в конструкторі форм графічний інтерфейс програми з віджетами QLabel, QLineEdit і QPushButton. *Використати додаткові віджети і елементи компоновки.

В. *Додати і відлагодити програмний код для введення вхідних даних з перевіркою на коректність (використати QMessageBox для виведення сповіщень), обчислень і виведення результатів.

С*. Додати пункти меню у QMenuBar для зчитування вхідних даних і збереження результатів в файл з використанням стандартних діалогів для вибору файлу.

Завдання 3. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів: 1) *«Ти - викладач, що приймає захист моєї роботи. Задай мені 5*

тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) *«Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання 2*

у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) *«Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»*

ВИКОНАННЯ РОБОТИ

Завдання 1 — Створення проекту десктоп-застосунку з використанням Qt Creator.

Алгоритм створення нового проекту у Qt Creator:

1. Натиснути клавішу «Create Project» на головному екрані Qt Creator;
2. Обрати тип проекту — Qt Widgets Application;
3. Обрати локацію проекту на диску;
4. Обрати систему збірки проекту — Cmake with Qt5 compatibility;
5. Обрати базовий клас (MainWindow) та відмітити пункт автоматичного створення форми;
6. Обрати файл перекладу, компілятор та дебаггер проекту;
7. Перевірити підсумок та натиснути «Finish».

Віджети, що використовуються у проєкті:

1. QMainWindow — віджет вікна програми. Основні властивості:
 - 1.1. iconSize — розмір іконок для панелі інструментів;
 - 1.2. toolButtonStyle — стиль кнопок на панелі інструментів;
 - 1.3. animated — анімація відкриття та закриття док-панелей;
 - 1.4. documentMode — оптимізація вкладок меню для MacOS;
 - 1.5. tabShape — форма вкладок для розділених док-панелей (закруглені або трикутні);
 - 1.6. dockNestingEnabled — дозволяє вкладення док-панелей;

1.7. `dockOptions` — додаткові параметри поведінки док-панелей.

2. `QPushButton` — кнопка. Основні властивості:

2.1. `text` — текст кнопки;

2.2. `icon` — іконка кнопки;

2.3. `iconSize` — розмір іконки;

2.4. `checkable` — режим перемикача;

2.5. `checked` — початковий стан кнопки в режимі перемикача;

2.6. `autoRepeat` — повторює натискання при затиску клавіші миші;

2.7. `autoExclusive` — режим радіо-кнопки для декількох кнопок в одному контейнері;

2.8. `default` — головна кнопка вікна за замовчанням, спрацьовує при натисканні `Enter` у вікні;

2.9. `flat` — плаский стиль відображення.

3. `QLabel` — мітка, статичний текст. Основні властивості:

3.1. `text` — текст мітки;

3.2. `pixmap` — зображення для мітки в режимі фоторамки;

3.3. `scaledContents` — масштабування зображення в режимі фоторамки;

3.4. `alignment` — вирівнювання тексту по вертикалі та горизонталі;

3.5. `wordWrap` — перенесення слів;

3.6. `textFormat` — формат тексту (простий, `html`, `markdown`);

3.7. `openExternalLinks` — відкривати посилання з тексту у браузері;

3.8. `buddy` — зв'язує мітку з іншим віджетом.

4. `QLineEdit` — однорядкове поле вводу. Основні властивості:

4.1. `text` — поточний текст поля;

4.2. `placeholderText` — текст-підказка, що відображається при порожньому полі;

4.3. `echoMode` — режим відображення тексту. `Normal` — текст без змін, `NoEcho` — введені символи не відображаються, `Password` — символ «*» замість літер, `PasswordEchoOnEdit` — літеру видно секунду після введення, а потім — символ «*»;

4.4. `maxLength` — максимальна кількість символів;

4.5. `readOnly` — заборона редагування поля;

4.6. `alignment` — вирівнювання тексту в полі;

4.7. `clearButtonEnabled` — додає клавішу для очищення поля;

4.8. `inputMask` — шаблон для введення даних.

5. `QMenuBar` — панель меню. Основні властивості:

5.1. `defaultUp` — напрямок, куди відкриваються випадаючі елементи;

5.2. `nativeMenuBar` — властивість для сумісності меню з системним рядком MacOS.

6. `QMenu` — вертикальне вікно з пунктами дій. Основні властивості:

6.1. `title` — текст заголовка меню;

6.2. `icon` — іконка меню;

6.3. `tearOffEnabled` — можливість «відірвати» меню від головного вікна;

6.4. `separatorsCollapsible` — ховати роздільні лінії, якщо між ними немає видимих пунктів меню.

7. `QAction` — дія пункту меню. Основні властивості:

- 7.1. `text` — назва дії в меню;
- 7.2. `icon` — іконка дії в меню;
- 7.3. `shortcut` — комбінація гарячих клавіш для виклику дії;
- 7.4. `checkable` — режим прапорця для пункту меню;
- 7.5. `checked` — стан режиму прапорця за замовчанням;
- 7.6. `enabled` — доступність дії;
- 7.7. `visible` — видимість дії;
- 7.8. `toolTip` — підказка при наведенні курсору;
- 7.9. `statusTip` — розгорнутий опис дії при наведенні курсору;
- 7.10. `whatsThis` — довідкова інформація щодо дії;
- 7.11. `font` — шрифт дії в меню;
- 7.12. `iconVisibleInMenu` — відображення іконки в меню;
- 7.13. `shortcutContext` — область дії гарячих клавіш.

Завдання 2 — Проект введення та виведення арифметичних операцій (Варіант 18).

З початку доби минуло N секунд (N — ціле). Знайти кількість секунд, що пройшли з початку останньої години.

Проект інтерфейсу представлено у графічному вигляді на рис. 1.

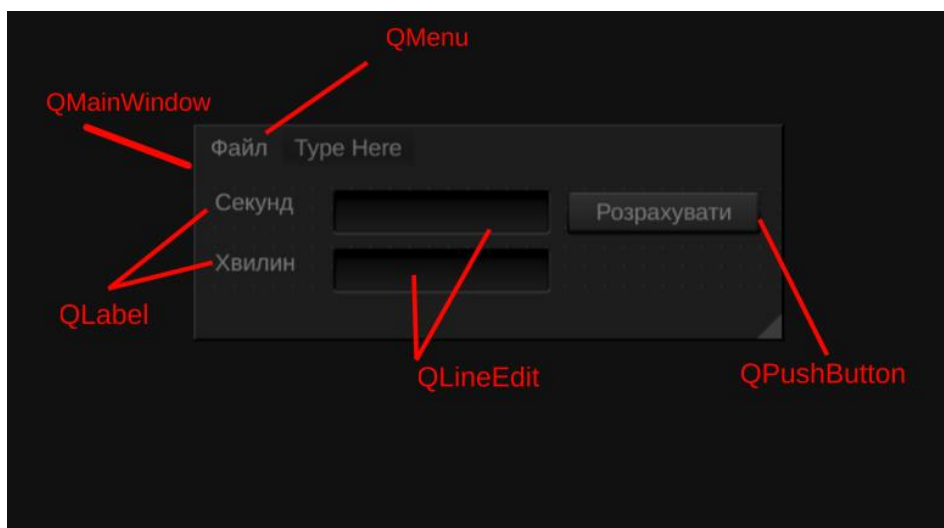


Рисунок 1 – Проект інтерфейсу

Код файла mainwindow.cpp:

```
#include "mainwindow.h"
#include "../ui_mainwindow.h"
#include <QMessageBox>
#include <QFileDialog>
#include <QDir>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_calcBtn_clicked()
{
    bool ok;
    int seconds = ui->secValue->text().toInt(&ok);
    if (!ok) {
        QMessageBox::warning(0, "Помилка", "Не вдалося зчитати число секунд");
        return;
    }
    if (seconds < 0) {
        QMessageBox::warning(0, "Помилка", "Кількість секунд не може бути менша за 0");
        return;
    }
}
```

```

    }
    int minutesTotal = seconds / 60;
    int minutesRemainder = minutesTotal % 60;
    ui->minValue->setText(QString::number(minutesRemainder));
}
void MainWindow::on_loadFile_triggered()
{
    QString filePath = QFileDialog::getOpenFileName(this, "Відкрити файл",
                                                    QDir::currentPath(),
"Текстові файли (*.txt);;Усі файли (*)");
    if (!filePath.isEmpty()) {
        QFile file(filePath);
        if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            QMessageBox::warning(0, "Error", "Не вдалося відкрити файл");
        }
        else {
            QTextStream in(&file);
            QString sec, min;
            in >> sec >> min;
            ui->secValue->setText(sec);
            ui->minValue->setText(min);
            file.close();
        }
    }
    else {
        QMessageBox::warning(0, "Error", "Файл не знайдено");
    }
}

void MainWindow::on_saveFile_triggered()
{
    QString filePath = QFileDialog::getSaveFileName(this, "Зберегти файл",
                                                    QDir::currentPath(),
"Текстові файли (*.txt);;Усі файли (*)");
    if (!filePath.isEmpty()) {
        QFile file(filePath);
        if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
            QMessageBox::critical(nullptr, "Помилка",
                                "Не вдалося відкрити файл:\n" +
file.errorString());
        }
        else {
            QTextStream out(&file);
            out << ui->secValue->text() << "\n";
            out << ui->minValue->text() << "\n";
            file.close();
        }
    }
    else {

```



```

    QMessageBox::warning(0, "Помилка", "Файл не знайдено");
}
}

```

Результати роботи десктоп-програми наведено на рис. 2.

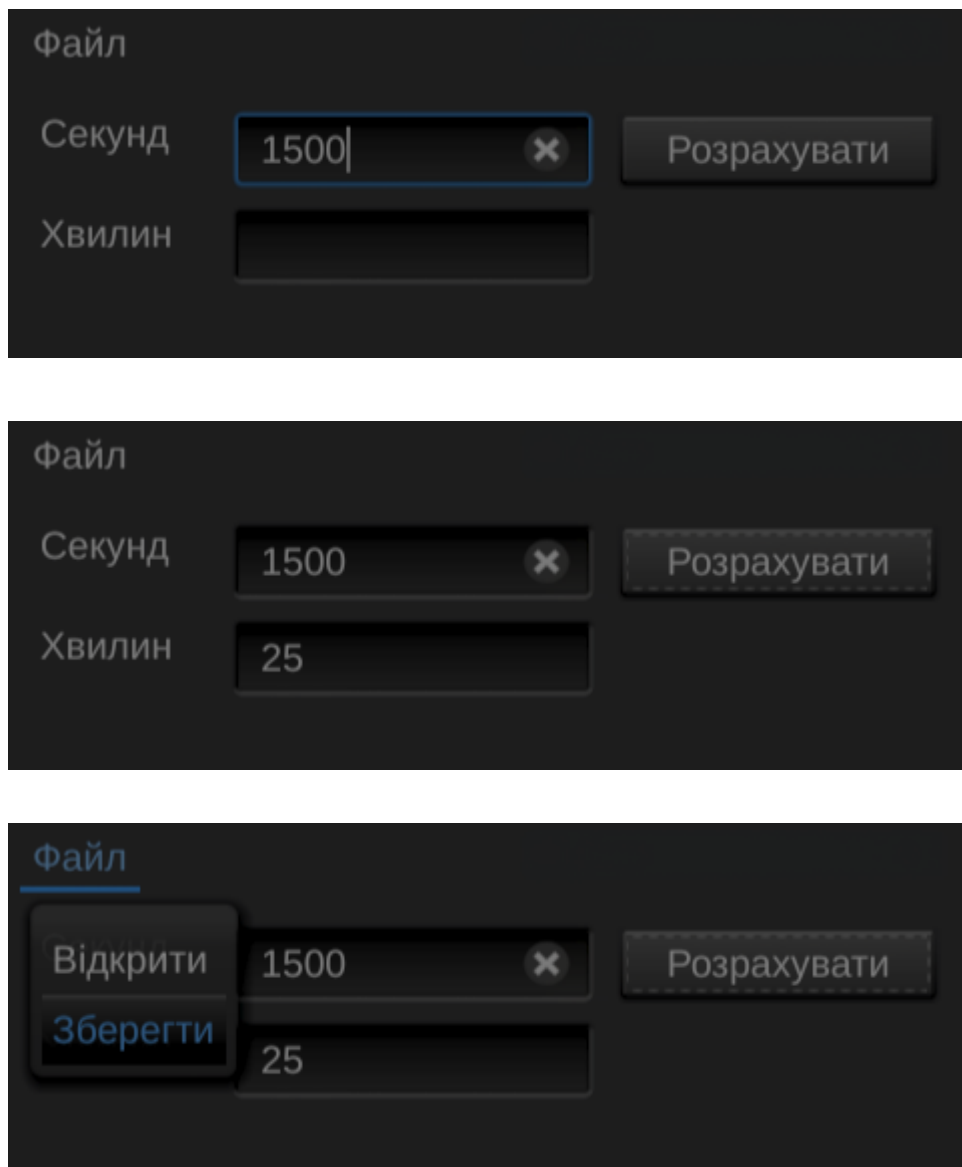


Рисунок 6 – Вікна програми при запуску

Лістинг коду вирішення задачі наведено в дод. А (стор. 11).

ВИСНОВКИ

Ознайомилися з основами розробки програм з використанням QtDesigner і навчитися розробляти десктоп-застосунки із графічним користувацьким інтерфейсом для введення/виведення даних на мові програмування C++ в середовищі QtCreator.

ДОДАТОК А

Лістинг коду програми

Файл **mainwindow.ui**

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
  <class>MainWindow</class>
  <widget class="QMainWindow" name="MainWindow">
    <property name="geometry">
      <rect>
        <x>0</x>
        <y>0</y>
        <width>301</width>
        <height>109</height>
      </rect>
    </property>
    <property name="windowTitle">
      <string>MainWindow</string>
    </property>
    <widget class="QWidget" name="centralwidget">
      <widget class="QLineEdit" name="secValue">
        <property name="geometry">
          <rect>
            <x>70</x>
            <y>10</y>
            <width>113</width>
            <height>24</height>
          </rect>
        </property>
        <property name="clearButtonEnabled">
          <bool>true</bool>
        </property>
      </widget>
      <widget class="QLabel" name="secLbl">
        <property name="geometry">
          <rect>
            <x>10</x>
            <y>10</y>
            <width>52</width>
            <height>14</height>
          </rect>
        </property>
        <property name="text">
          <string>Секунд</string>
        </property>
      </widget>
    </widget>
  </widget>
</ui>
```

```

</widget>
<widget class="QPushButton" name="calcBtn">
  <property name="geometry">
    <rect>
      <x>190</x>
      <y>10</y>
      <width>101</width>
      <height>24</height>
    </rect>
  </property>
  <property name="text">
    <string>Розрахувати</string>
  </property>
</widget>
<widget class="QLabel" name="minLbl">
  <property name="geometry">
    <rect>
      <x>10</x>
      <y>40</y>
      <width>51</width>
      <height>16</height>
    </rect>
  </property>
  <property name="text">
    <string>Хвилин</string>
  </property>
</widget>
<widget class="QLineEdit" name="minValue">
  <property name="geometry">
    <rect>
      <x>70</x>
      <y>40</y>
      <width>113</width>
      <height>24</height>
    </rect>
  </property>
  <property name="readOnly">
    <bool>true</bool>
  </property>
</widget>
</widget>
<widget class="QMenuBar" name="menubar">
  <property name="geometry">
    <rect>
      <x>0</x>
      <y>0</y>
      <width>301</width>
      <height>22</height>
    </rect>
  </property>

```

```

<widget class="QMenu" name="loadFileMenuItem">
  <property name="title">
    <string>Файл</string>
  </property>
  <addaction name="loadFile"/>
  <addaction name="saveFile"/>
</widget>
<addaction name="loadFileMenuItem"/>
</widget>
<widget class="QStatusBar" name="statusbar"/>
<action name="loadFile">
  <property name="enabled">
    <bool>true</bool>
  </property>
  <property name="icon">
    <iconset theme="QIcon::ThemeIcon::DocumentOpen"/>
  </property>
  <property name="text">
    <string>Відкрити</string>
  </property>
  <property name="shortcutContext">
    <enum>Qt::ShortcutContext::ApplicationShortcut</enum>
  </property>
  <property name="menuRole">
    <enum>QAction::MenuRole::NoRole</enum>
  </property>
</action>
<action name="saveFile">
  <property name="icon">
    <iconset theme="QIcon::ThemeIcon::DocumentSave"/>
  </property>
  <property name="text">
    <string>Зберегти</string>
  </property>
  <property name="menuRole">
    <enum>QAction::MenuRole::NoRole</enum>
  </property>
</action>
</widget>
<resources/>
<connections/>
</ui>

```

Файл **mainwindow.h**

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

```

```

#include <QMainWindow>

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow() override;

private slots:
    void on_calcBtn_clicked();

    void on_loadFile_triggered();

    void on_saveFile_triggered();

private:
    Ui::MainWindow *ui;
};
#endif // MAINWINDOW_H

```

Файл **mainwindow.cpp**

```

#include "mainwindow.h"
#include "../ui_mainwindow.h"
#include <QMessageBox>
#include <QFileDialog>
#include <QDir>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}

```

```

void MainWindow::on_calcBtn_clicked()
{
    bool ok;
    int seconds = ui->secValue->text().toInt(&ok);
    if (!ok) {
        QMessageBox::warning(0, "Помилка", "Не вдалося зчитати число секунд");
        return;
    }
    if (seconds < 0) {
        QMessageBox::warning(0, "Помилка", "Кількість секунд не може бути менша за 0");
        return;
    }
    int minutesTotal = seconds / 60;
    int minutesRemainder = minutesTotal % 60;
    ui->minValue->setText(QString::number(minutesRemainder));
}

void MainWindow::on_loadFile_triggered()
{
    QString filePath = QFileDialog::getOpenFileName(this, "Відкрити файл",
                                                    QDir::currentPath(),
                                                    "Текстові файли (*.txt);;Усі файли (*)");
    if (!filePath.isEmpty()) {
        QFile file(filePath);
        if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            QMessageBox::warning(0, "Error", "Не вдалося відкрити файл");
        }
        else {
            QTextStream in(&file);
            QString sec, min;
            in >> sec >> min;
            ui->secValue->setText(sec);
            ui->minValue->setText(min);
            file.close();
        }
    }
    else {
        QMessageBox::warning(0, "Error", "Файл не знайдено");
    }
}

void MainWindow::on_saveFile_triggered()
{
    QString filePath = QFileDialog::getSaveFileName(this, "Зберегти файл",
                                                    QDir::currentPath(),
                                                    "Текстові файли (*.txt);;Усі файли (*)");
    if (!filePath.isEmpty()) {
        QFile file(filePath);
    }
}

```

```

        if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
            QMessageBox::critical(nullptr, "Помилка",
                                   "Не вдалося відкрити файл:\n" +
file.errorString());
        }
        else {
            QTextStream out(&file);
            out << ui->secValue->text() << "\n";
            out << ui->minValue->text() << "\n";
            file.close();
        }
    }
    else {
        QMessageBox::warning(0, "Помилка", "Файл не знайдено");
    }
}

```

Файл **main.cpp**

```

#include "mainwindow.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return QApplication::exec();
}

```


ДОДАТОК В

Діалог з ШІ Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції

```
Файл mainwindow.ui <?xml version="1.0" encoding="UTF-8"?> <ui version="4.0"> <class>Main-
Window</class> <widget class="QMainWindow" name="MainWindow"> <property name="ge-
ometry"> <rect> <x>0</x> <y>0</y> <width>301</width> <height>109</height> </rect> </prop-
erty> <property name="windowTitle"> <string>MainWindow</string> </property> <widget
class="QWidget" name="centralwidget"> <widget class="QLineEdit" name="secValue"> <prop-
erty name="geometry"> <rect> <x>70</x> <y>10</y> <width>113</width> <height>24</height>
</rect> </property> <property name="clearButtonEnabled"> <bool>true</bool> </property>
</widget> <widget class="QLabel" name="secLbl"> <property name="geometry"> <rect>
<x>10</x> <y>10</y> <width>52</width> <height>14</height> </rect> </property> <property
name="text"> <string>Секунд</string> </property> </widget> <widget class="QPushButton"
name="calcBtn"> <property name="geometry"> <rect> <x>190</x> <y>10</y>
<width>101</width> <height>24</height> </rect> </property> <property name="text">
<string>Розрахувати</string> </property> </widget> <widget class="QLabel" name="minLbl">
<property name="geometry"> <rect> <x>10</x> <y>40</y> <width>51</width>
<height>16</height> </rect> </property> <property name="text"> <string>Хвилин</string>
</property> </widget> <widget class="QLineEdit" name="minValue"> <property name="geome-
try"> <rect> <x>70</x> <y>40</y> <width>113</width> <height>24</height> </rect> </prop-
erty> <property name="readOnly"> <bool>true</bool> </property> </widget> </widget> <widget
class="QMenuBar" name="menubar"> <property name="geometry"> <rect> <x>0</x> <y>0</y>
<width>301</width> <height>22</height> </rect> </property> <widget class="QMenu"
name="loadFileMenuItem"> <property name="title"> <string>Файл</string> </property> <addac-
tion name="loadFile"/> <addaction name="saveFile"/> </widget> <addaction name="loadFile-
MenuItem"/> </widget> <widget class="QStatusBar" name="statusbar"/> <action name="load-
File"> <property name="enabled"> <bool>true</bool> </property> <property name="icon">
<iconset theme="QIcon::ThemeIcon::DocumentOpen"/> </property> <property name="text">
<string>Відкрити</string> </property> <property name="shortcutContext">
<enum>Qt::ShortcutContext::ApplicationShortcut</enum> </property> <property name="menu-
Role"> <enum>QAction::MenuRole::NoRole</enum> </property> </action> <action name="save-
File"> <property name="icon"> <iconset theme="QIcon::ThemeIcon::DocumentSave"/> </prop-
erty> <property name="text"> <string>Зберегти</string> </property> <property name="menu-
Role"> <enum>QAction::MenuRole::NoRole</enum> </property> </action> </widget> <re-
sources/> <connections/> </ui> Файл mainwindow.h #ifndef MAINWINDOW_H #define MAIN-
WINDOW_H #include <QMainWindow> QT_BEGIN_NAMESPACE namespace Ui { class Main-
Window; } QT_END_NAMESPACE class MainWindow : public QMainWindow { Q_OBJECT
public: explicit MainWindow(QWidget *parent = nullptr); ~MainWindow() override; private slots:
void on_calcBtn_clicked(); void on_loadFile_triggered(); void on_saveFile_triggered(); private:
Ui::MainWindow *ui; }; #endif // MAINWINDOW_H Файл mainwindow.cpp #include "main-
window.h" #include "./ui_mainwindow.h" #include <QMessageBox> #include <QFileDialog> #in-
clude <QDir> MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) , ui(new
Ui::MainWindow) { ui->setupUi(this); } MainWindow::~~MainWindow() { delete ui; } void Main-
Window::on_calcBtn_clicked() { bool ok; int seconds = ui->secValue->text().toInt(&ok); if (!ok) {
QMessageBox::warning(0,"Помилка","Не вдалося зчитати число секунд"); return; } if (seconds
< 0) { QMessageBox::warning(0,"Помилка","Кількість секунд не може бути менша за 0"); re-
turn; } int minutesTotal = seconds / 60; int minutesRemainder = minutesTotal % 60; ui->minValue-
>setText(QString::number(minutesRemainder)); } void MainWindow::on_loadFile_triggered() {
```

```
QString filePath = QFileDialog::getOpenFileName(this, "Відкрити файл", QDir::currentPath(),
"Текстові файли (*.txt); Усі файли (*)"); if (!filePath.isEmpty()) { QFile file(filePath); if
(!file.open(QIODevice::ReadOnly | QIODevice::Text)) { QMessageBox::warning(0, "Error", "Не
вдалося відкрити файл"); } else { QTextStream in(&file); QString sec, min; in >> sec >> min; ui-
>secValue->setText(sec); ui->minValue->setText(min); file.close(); } } else { QMessage-
Box::warning(0, "Error", "Файл не знайдено"); } } void MainWindow::on_saveFile_triggered() {
QString filePath = QFileDialog::getSaveFileName(this, "Зберегти файл", QDir::currentPath(),
"Текстові файли (*.txt); Усі файли (*)"); if (!filePath.isEmpty()) { QFile file(filePath); if
(!file.open(QIODevice::WriteOnly | QIODevice::Text)) { QMessageBox::critical(nullptr,
"Помилка", "Не вдалося відкрити файл:\n" + file.errorString()); } else { QTextStream out(&file);
out << ui->secValue->text() << "\n"; out << ui->minValue->text() << "\n"; file.close(); } } else {
QMessageBox::warning(0, "Помилка", "Файл не знайдено"); } } Файл main.cpp #include "main-
window.h" #include <QApplication> int main(int argc, char *argv[]) { QApplication a(argc, argv);
MainWindow w; w.show(); return QApplication::exec(); }
```

Тестові питання

1. Яка головна проблема є в логіці обчислення хвилин у методі

`on_calcBtn_clicked()` ?

- A) Не враховуються від'ємні числа
- B) Результат може переповнити `QString`
- C) Обчислюються хвилини в межах години, а не загальна кількість хвилин
- D) Використовується неправильний тип даних

2. Що станеться, якщо у файл для завантаження записати лише одне число замість двох?

- A) Програма аварійно завершиться
- B) `minValue` залишиться порожнім або збереже старе значення
- C) `QTextStream` автоматично допише друге число
- D) `QFile` не дозволить прочитати файл

3. Для чого в класі `mainwindow` використовується макрос `Q_OBJECT`?

- A) Для автоматичного створення вікна
- B) Для підтримки механізму сигналів і слотів Qt
- C) Для роботи з файловою системою
- D) Для створення меню

4. Яка проблема можлива при використанні `toInt(&ok)` у цьому коді?

- A) Метод не може обробляти великі числа
- B) Метод повертає `QString`
- C) Значення, що виходять за межі `int`, можуть бути оброблені некоректно
- D) `toInt()` не працює з `QLineEdit`

5. Чому поле `minValue` зроблено `readOnly=true` у `.ui` файлі?

- A) Щоб заборонити користувачу змінювати результат обчислень вручну
- B) Щоб прискорити роботу програми

- C) Щоб `QTextStream` міг записувати у поле
D) Щоб приховати значення від користувача

Відкриті питання

1.

Проаналізуйте метод `on_calcBtn_clicked()`.

Які логічні або інженерні недоліки ви бачите в обчисленні хвилин і як би ви переписали цей фрагмент?

2.

У методах `on_loadFile_triggered()` та `on_saveFile_triggered()` використовується робота з файлами.

Які перевірки або механізми варто додати, щоб зробити роботу з файлами більш надійною та безпечною?

3.

Поясніть, як у Qt працює механізм сигналів і слотів на прикладі натискання кнопки `calcBtn`.

4.

У коді використовуються як `nullptr`, так і `0` при виклику `QMessageBox`.

Чому сучасний C++ рекомендує використовувати `nullptr` замість `0` або `NULL`?

5.

Як би ви змінили архітектуру цієї програми, якби потрібно було:

- підтримувати години, хвилини та секунди,
- зберігати дані у JSON,
- додати автоматичну перевірку введення під час набору тексту?

Відповіді на тестові запитання: 1) С - Обчислюються хвилини в межах години, а не загальна кількість хвилин 2) В - `minValue` залишиться порожнім або збереже старе значення 3) В - Для підтримки механізму сигналів і слотів Qt 4) С - Значення, що виходять за межі `int`, можуть оброблятися некоректно 5) А - Щоб заборонити користувачам змінювати результат обчислень вручну

Відповіді на відкриті питання: 1) Головна проблема - можливість переповнення значення секунд. Щоб уникнути цього, слід було б налаштувати максимальне значення вводу для текстового поля. За умовою задачі, слід було розрахувати кількість хвилин з останньої години, тож значення хвилин розраховується правильно, проте якщо б була потреба розрахувати повну кількість хвилин в заданих секундах, можна було б повертати як результат `minutesTotal` замість `minutesRemainder` 2) Основні перевірки, що слід було б додати - це перевірка на кількість введених елементів (завжди має бути 2 значення - `sec` та `min`), та перевірка на правильність цих значень, тобто `sec` мають бути позитивним числом (додатково можна перевірити розмір для використання числа як `int`), `min` мають бути позитивним числом

зі значенням від 0 до 59 3) Сигнали та слоти - це основний механізм "динамічності" Qt. Віджети Qt надсилають сигнали при зміні стану, а за обробку цих сигналів відповідають слоти. Розглядаючи нашу кнопку, при натисканні надсилається сигнал типу "clicked", який підхоплюється функцією `on_calcBtn_clicked()`. Таким чином, сигнали та слоти дозволяють нам реагувати на дії користувача програми 4) При виклику `QMessageBox` є два варіанти першого параметру: `QWidget *parent` або `int errorCode`. При використанні 0, ми фактично ігноруємо батьківське вікно, що може привести до незручностей, наприклад, коли вікно повідомлення сховається за батьківським та програма умовно "зависне". 5) Для підтримки хвилин, годин та секунд, слід було б додати ще одне поле вводу з міткою для годин, та модифікувати функцію `on_calcBtn_clicked()` для додаткового визначення кількості годин. Для збереження в JSON слід було б використати модуль `QJson`, створити об'єкт `QJsonDocument`, задати його властивості та записати у файл використовуючи `doc.toJson()`. Автоматичну перевірку можна було б використати вбудований валідатор типу `QIntValidator`, що дозволило б задати діапазон значень, або використовувати регулярні вирази. Також можна використати сигнали зміни тексту (`textChanged`, `textEdited`, `editingFinished`), де перевіряти значення вручну або застосовувати валідатори, вказані вище.

Загалом відповіді хороші — видно, що ти розумієш як Qt, так і базові принципи проєктування GUI-застосунків. Більшість відповідей не просто "вгадані", а аргументовані, що важливо на захисті.

Оцінка тестових питань

1. ✓ Правильно
2. ✓ Правильно
3. ✓ Правильно
4. ✓ Правильно
5. ✓ Правильно

Результат: 5/5

Коментарі до відкритих питань

1. Аналіз `on_calcBtn_clicked()`

✓ Хороша відповідь.

Ти правильно помітив:

- можливість переповнення,
- важливість обмеження вводу,
- залежність логіки від постановки задачі.

Особливо добре, що ти звернув увагу на формулювання задачі — це вже інженерне мислення, а не лише "читання коду".

Що ще можна було додати:

- `QIntValidator` краще за "ручні перевірки" для GUI,
- `minutesRemainder` можна було б назвати зрозуміліше (`minutesWithinHour`).

2. Робота з файлами

✓ Дуже хороша відповідь.

Ти охопив:

- перевірку формату,
- валідацію даних,
- контроль діапазонів.

Це саме те, чого очікують на захисті.

Що ще можна було згадати:

- обробку порожнього файлу,
- повідомлення користувачу про конкретну помилку,
- використання `QSaveFile` для безпечного збереження.

3. Сигнали і слоти

✓ Правильно і достатньо повно.

Особливо добре:

- пояснення подієвої моделі,
- зв'язок із GUI,
- приклад з `clicked`.

Невелике уточнення:

у твоєму випадку слот викликається автоматично через `naming convention` (`on_<object>_<signal>`), яку генерує Qt Designer.

4. `nullptr` VS 0

⚠ Тут є неточність.

Ти правильно сказав про проблему з батьківським вікном, але пояснення про “два варіанти параметру” некоректне для цього виклику.

Головна причина використання `nullptr`:

- `nullptr` має окремий тип (`std::nullptr_t`),
- він однозначно означає “нульовий вказівник”,
- 0 — це просто ціле число, що може викликати неоднозначності при перевантаженні функцій.

Тобто проблема тут більше в **типобезпеці**, ніж у логіці `QMessageBox`.

5. Розширення архітектури

✓ Сильна відповідь.

Особливо добре:

- згадка про `QJsonDocument`,
- використання валідаторів,

- перелік сигналів для реакції на введення.

Що ще можна було б додати:

- винесення логіки обчислень в окремий клас,
- розділення GUI та бізнес-логіки,
- використання структури/моделі для часу.

Підсумок

Твій рівень відповідей:

- впевнене розуміння Qt Widgets,
- хороше знання сигналів/слотів,
- адекватне розуміння валідації та роботи з файлами,
- є інженерне мислення, а не лише знання синтаксису.

Найслабше місце:

- тонкощі сучасного C++ (`nullptr`, типобезпека, `overload resolution`).

Якби це був реальний захист лабораторної — це виглядало б приблизно на **добрий/відмінний рівень** залежно від вимог викладача.

я самоаналізу